Nume și prenume:

Grupă:

Examen POO

A. Test grilă (30 X 1pct)

1. Care este rezultatul următoarei secvențe?

```
1. package pkgA;
2. public class Foo {
3.   int a = 5;
4.   protected int b = 6;
5.   public int c = 7;
6. }
3. package pkgB;
4. import pkgA.*;
5. public class Baz {
6.   public static void main(String[] args) {
7.     Foo f = new Foo();
8.     System.out.print(" " + f.a);
9.     System.out.print(" " + f.b);
10.    System.out.print(" " + f.c);
11.  }
12. }
```

A. 5 6 7
B. 5 urmat de eroare
C. Eroare de compilare la linia 8
D. Eroare de compilare la linia 10

2. Care este rezultatul următoarei secvențe?

```
4. class Announce {
5.   public static void main(String[] args) {
6.     for(int __x = 0; __x < 3; __x++) ;
7.     int #lb = 7;
8.     long [] x [5];
9.     Boolean []ba[];
10.    enum Traffic { RED, YELLOW, GREEN };
11.   }
12. }
```

A. Compilare cu succes
B. Eroare de compilare la linia 6
C. Eroare de compilare la linia 8
D. Eroare de compilare la linia 10

3. Care este rezultatul următoarei secvențe?

```
class Top {
  public Top(String s) { System.out.print("B"); }
}
public class Bottom2 extends Top {
  public Bottom2(String s) { System.out.print("D");}
  public static void main(String [] args) {
    new Bottom2("C");
    System.out.println(" ");
} }
```

A. Eroare de compilare
B. BD
C. DB
D. BDC

4. Care afirmatie este corecta?

A. Coeziunea este un principiu de programare orientata obiect asociat cu ascunderea detaliilor de implementare
B. Coeziunea este un principiu de programare orientata obiect prin care functionalitatile unei clase sunt expuse printr-un API
C. Coeziunea este un principiu de programare orientata obiect prin care scopul unei clase se rezuma exclusiv la un punct de interes bine definit
D. Coeziunea este un principiu de programare orientata obiect prin care un obiect poate fi perceput ca avand multiple tipuri

5. Care este rezultatul următoarei secvențe?

```
1. class MyException extends RuntimeException { }
2. public class Houdini {
3.   public static void main(String[] args) throws
Exception {
4.     throw new MyException();
5.     System.out.println("success");
6. } }
```

A. succes
B. Eroare de compilare la linia 3
C. Eroare de compilare la linia 4
D. Eroare de compilare la linia 5

6. Care este rezultatul următoarei secvențe?

```
3. class Alpha {
4.   static String s = " ";
5.   protected Alpha() { s += "alpha "; }
6. }
7. class SubAlpha extends Alpha {
8.   private SubAlpha() { s += "sub "; }
9. }
10. public class SubSubAlpha extends Alpha {
11.   private SubSubAlpha() { s += "subsub "; }
12.   public static void main(String[] args) {
13.     new SubSubAlpha();
14.     System.out.println(s);
15.   }
16. }
```

A. subsub
B. sub subsub
C. alpha subsub
D. Eroare de compilare

7. Cate obiecte sunt eligibile sa fie colectate de GC la atingerea secventei indicate?

```
class CardBoard {
  Short story = 200;
  CardBoard go(CardBoard cb) {
    cb = null;
    return cb;
  }
  public static void main(String[] args) {
    CardBoard c1 = new CardBoard();
    CardBoard c2 = new CardBoard();
    CardBoard c3 = c1.go(c2);
    c1 = null;
    // secventa rulare GC
} }
```

A. 0
B. 1
C. 2
D. Imposibil de determinat cu exactitate

8. Care eroare poate fi prinsa in cadrul urmatorului bloc?

```
try { int x = Integer.parseInt("unu"); }
```

A. NumberFormatException
B. ClassCastException
C. ArrayIndexOutOfBoundsException
D. IllegalStateException

9. Care este rezultatul următoarei secvențe?

```
3. public class KaChung {
4.   public static void main(String[] args) {
5.     String s = "";
6.     if(Integer.parseInt("011") ==
Integer.parseInt("9")) s += 1;
7.     if(021 == Integer.valueOf("17")) s += 2;
8.     if(1024 == new Integer(1024)) s += 3;
9.     System.out.println(s);
10. } }
```

A. 2
B. 3
C. 13
D. 23

10. Care este rezultatul următoarei secvențe?

```
3. public static void main(String[] args) {
4.   try {
5.    throw new Error();
6.   }
7.   catch (Error e) {
8.     try { throw new RuntimeException(); }
9.     catch (Throwable t) { }
10.  }
11.  System.out.println("oioioi");
12. }
```

A. oioioi
B. Eroare de compilare la linia 5
C. Eroare de compilare la linia 8
D. Eroare de compilare la linia 9

11. Care este rezultatul următoarei secvențe?

```
class Bird {
 { System.out.print("b1 "); }
  public Bird() { System.out.print("b2 "); }
}
class Raptor extends Bird {
  static { System.out.print("r1 "); }
  public Raptor() { System.out.print("r2 "); }
   { System.out.print("r3 "); }
  static { System.out.print("r4 "); }
}
class Hawk extends Raptor {
  public static void main(String[] args) {
    System.out.print("pre ");
    new Hawk();
    System.out.println("hawk ");
  }
}
```

A. pre b1 b2 r3 r2 hawk
B. pre b2 b1 r2 r3 hawk
C. pre b2 b1 r2 r3 hawk r1 r4
D. r1 r4 pre b1 b2 r3 r2 hawk

12. Care este rezultatul următoarei secvențe?

```
3. public class Ouch {
4.   static int ouch = 7;
5.   public static void main(String[] args) {
6.     new Ouch().go(ouch);
7.     System.out.print(" " + ouch);
8.   }
9.   void go(int ouch) {
10.    ouch++;
11.    for(int ouch = 3; ouch < 6; ouch++)
12.    ;
13.    System.out.print(" " + ouch);
14.    }
15. }
```

A. Eroare de compilare
B. 5 7
C. 5 8
D. 8 8

13. Care afirmatie este corecta in momnetul atingerii liniei 14?

```
3. class Dozens {
4.   int[] dz = {1,2,3,4,5,6,7,8,9,10,11,12};
5. }
6. public class Eggs {
7.   public static void main(String[] args) {
8.     Dozens [] da = new Dozens[3];
9.     da[0] = new Dozens();
10.    Dozens d = new Dozens();
11.    da[1] = d;
12.    d = null;
13.    da[1] = null;
14. // secventa rulare GC
15. }
16. }
```

A. 1 obiect este preluat de GC
B. 2 obiecte sunt preluate de GC
C. 3 obiecte au fost create
D. 4 obiecte au fost create

14. Pornind de la declaratia?

```
1. class Comp {
2.   public static void main(String[] args) {
3.    float f1 = 2.3f;
4.    float[][] f2 = {{42.0f}, {1.7f, 2.3f}, {2.6f,
2.7f}};
5.    float[] f3 = {2.7f};
6.    Long x = 42L;
7.    // inserati o secventa de cod aici
8.    System.out.println("true");
9.   }
10.}
```

Si avand urmatoarele secvente de cod:

```
F1. if(f1 == f2)
F2. if(f1 == f2[2][1])
F3. if(x == f2[0][0])
F4. if(f1 == f2[1,1])
F5. if(f3 == f2[2])
```

Care afirmatie este corecta

A. 1 secv va compila si 1 secv va intoarce true
B. 2 secv vor compila si 1 secv va intoarce true
C. 3 secv vor compila si 1 secv va intoarce true
D. 3 secv vor compila si 2 secv vor intoarce true

15. Care este rezultatul următoarei secvențe?

```
3. interface Vessel { }
4. interface Toy { }
5. class Boat implements Vessel { }
6. class Speedboat extends Boat implements Toy { }
7. public class Tree {
8.   public static void main(String[] args) {
9.     String s = "0";
10.   Boat b = new Boat();
11.   Boat b2 = new Speedboat();
12.   Speedboat s2 = new Speedboat();
13.   if((b instanceof Vessel) && (b2 instanceof
Toy)) s += "1";
14.   if((s2 instanceof Vessel) && (s2 instanceof
Toy)) s += "2";
15.    System.out.println(s);
16.   }
17. }
```

A. 0
B. 01
C. 02
D. 012

16. Care este rezultatul următoarei secvențe?

```
3. public class Circles {
4.   public static void main(String[] args) {
5.     int[] ia = {1,3,5,7,9};
6.     for(int x : ia) {
7.       for(int j = 0; j < 3; j++) {
8.         if(x > 4 && x < 8) continue;
9.         System.out.print(" " + x);
10.        if(j == 1) break;
11.        continue;
12.      }
13.      continue;
14.    }
15.  }
16. }
```

A. 1 3 9
B. 5 5 7 7
C. 1 3 3 9 9
D. 1 1 3 3 9 9

17. Care afirmatie este corecta?

```
3. public class Theory {
4.   public static void main(String[] args) {
5.     String s1 = "abc";
6.     String s2 = s1;
7.     s1 += "d";
8.     System.out.println(s1 + " " + s2 + " " +
(s1==s2));
9.
10.    StringBuffer sb1 = new StringBuffer("abc");
11.    StringBuffer sb2 = sb1;
12.    sb1.append("d");
13.    System.out.println(sb1 + " " + sb2 + " " +
(sb1==sb2));
14.  }
15. }
```

A. Prima linie este abc abc false
B. Prima linie este abcd abc false
C. A doua linie este abc abc false
D. A doua linie este abcd abc false

18. Care declaratie este admisibila?

```
public static void main(String[] args) {
  // inserati declaratia
  for (int i = 0; i <= 10; i++) {
    List<Integer> row = new ArrayList<Integer>();
    for (int j = 0; j <= 10; j++)
      row.add(i * j);
    table.add(row);
  }
  for (List<Integer> row : table)
    System.out.println(row);
}
```

A. List<List<Integer>> table = new List<List<Integer>>();
B. List<List<Integer>> table = new ArrayList<List<Integer>>();
C. List<List<Integer>> table = new ArrayList<ArrayList<Integer>>();
D. List<List, Integer> table = new List<List, Integer>();

19. Care este rezultatul următoarei secvențe?

```
3. import java.util.*;
4. class Dog { int size; Dog(int s) { size = s; } }
5. public class FirstGrade {
6.   public static void main(String[] args) {
7.     TreeSet<Integer> i = new TreeSet<Integer>();
8.     TreeSet<Dog> d = new TreeSet<Dog>();
9.
10.    d.add(new Dog(1)); d.add(new Dog(2));
d.add(new Dog(1));
11.    i.add(1); i.add(2); i.add(1);
12.    System.out.println(d.size() + " " + i.size());
13.  }
14. }
```

A. 1 2
B. 2 2
C. 2 3
D. Exceptie de rulare

20. Care afirmatie este adevarata despre o metoda a unei clase interne?

A. Trebuie sa fie marcata final
B. Poate fi marcata abstract
C. Trebuie sa fie marcata public
D. Poate fi marcata static

21. Care este rezultatul următoarei secvențe?

```
1. public class HorseTest {
2.   public static void main(String[] args) {
3.     class Horse {
4.       public String name;
5.       public Horse(String s) {
6.         name = s;
7.       }
8.     }
9.     Object obj = new Horse("Zippo");
10.   System.out.println(obj.name);
11.   }
12. }
```

A. Exceptie de rulare la linia 10
B. Eroare de compilare la linia 3
C. Zippo
D. Eroare de compilare la linia 10

22. Care este rezultatul următoarei secvențe?

```
5. class A { void m() { System.out.println("outer");
} }
6. public class TestInners {
7.   public static void main(String[] args) {
8.     new TestInners().go();
9.   }
10.  void go() {
11.    new A().m();
12.    class A { void m() {
System.out.println("inner"); } }
13.   }
14.  class A { void m() {
System.out.println(“middle”); } }
15. }
```

A. inner
B. middle
C. outer
D. Exceptie la rulare

23. Care este rezultatul următoarei secvențe?

```
3. public class Navel {
4.   private int size = 7;
5.   private static int length = 3;
6.   public static void main(String[] args) {
7.    new Navel().go();
8.   }
9.   void go() {
10.   int size = 5;
11.   System.out.println(new Gazer().adder());
12.  }
13.  class Gazer {
14.    int adder() { return size * length; }
15.  }
16. }
```

A. 15
B. 21
C. Eroare de compilare la linia 10
D. Exceptie la rulare

24. Care este rezultatul următoarei secvențe?

```
class MyThread extends Thread {
  MyThread() {
    System.out.print(" MyThread");
  }
  public void run() { System.out.print(" bar"); }
  public void run(String s) { System.out.print("
baz"); }
}
public class TestThreads {
  public static void main (String [] args) {
    Thread t = new MyThread() {
      public void run() { System.out.print(" foo"); }
    };
    t.start();
} }
```

A. foo
B. MyThread foo
C. MyThread bar
D. foo bar

25. Care este rezultatul următoarei secvențe?

```
1. public class Argue {
2.   static boolean b;
3.   static int x = 0;
4.   public static void main(String[] args) {
5.     int guess = (int)(Math.random() * 5);
6.     if(guess < 0) assert false;
7.     assert b = true;
8.     assert x = 0;
9.     assert x == 0;
10. } }
```

A. Eroare de compilare la linia 6
B. Eroare de compilare la linia 7
C. Eroare de compilare la linia 8
D. Eroare de compilare la linia 9

26. Care este rezultatul următoarei secvențe?

```
2. import java.util.*;
3. public class AndOver {
4.   public static void main(String[] args) {
5.    List g = new ArrayList();
6.    g.add(new Gaited("Eyra"));
7.    g.add(new Gaited("Vafi"));
8.    g.add(new Gaited("Andi"));
9.    Iterator i2 = g.iterator();
10.  while(i2.hasNext()) {
11.    System.out.print(i2.next().name + " ");
12.  } } }
13. class Gaited {
14.   public String name;
15.   Gaited(String n) { name = n; }
16. }
```

A. Eroare de rulare
B. Andi Eyra Vafi
C. Eyra Vafi Andi
D. Eroare de compilare

27. Care este rezultatul următoarei secvențe?

```
2. public class Maize {
3.   public static void main(String[] args) {
4.     String s = "12";
5.     s.concat("ab");
6.     s = go(s);
7.     System.out.println(s);
8.   }
9.   static String go(String s) {
10.   s.concat("56");
11.   return s;
12. } }
```

A. ab
B. 12
C. ab56
D. 12ab56

28. Care este rezultatul următoarei secvențe?

```
13. int count = 0;
14. outer:
15. for(int x = 0; x < 5; x++) {
16.   middle:
17.   for(int y = 0; y < 5; y++) {
18.     if(y == 1) continue middle;
19.     if(y == 3) break middle;
20.     count++;
21.   }
22.   if(x > 2) continue outer;
23.   count = count + 10;
24. }
25. System.out.println("count: " + count);
```

A. count: 33
B. count: 40
C. count: 45
D. count: 65

29. Care este rezultatul următoarei secvențe?

```
5. boolean[] ba = {true, false};
6. short[][] gr = {{1,2}, {3,4}};
7. int i = 0;
8. for( ; i < 10; ) i++;
9. for(short s: gr) ;
10. for(int j = 0, k = 10; k > j; ++j, k--) ;
11. for(int j = 0; j < 3; System.out.println(j++)) ;
12. for(Boolean b: ba) ;
```

A. Eroare de compilare la linia 8
B. Eroare de compilare la linia 9
C. Eroare de compilare la linia 10
D. Eroare de compilare la linia 11

30. Care este rezultatul următoarei secvențe?

```
2. public class Clover extends Harrier {
3.   String bark() { return "feed me "; }
4.   public static void main(String[] args) {
5.     Dog[] dogs = new Dog[3];
6.     dogs[0] = new Harrier();
7.     dogs[1] = (Dog)new Clover();
8.     dogs[2] = (Dog)new Harrier();
9.     for(Dog d: dogs) System.out.print(d.bark());
10. } }
11. class Dog { String bark() { return "bark "; } }
12. class Harrier extends Dog { String bark() {
return "woof "; } }
```

A. bark bark bark
B. woof bark bark
C. woof feed me woof
D. Eroare de compilare la linia 6.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A															
B															
C															
D															
	16	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
A															
B															
C															
D															